



Go Game Project Report

By ABDELFATAH M. A. ALHOOT 2321051372

1. Project Understanding

This project is a board game **Go** (9x9 board), built using Java with network and multiplayer sessions feature. It uses a client-server architecture. The user interface is developed using Java Swing (NetBeans GUI Builder), and the networking relies on TCP sockets communicating through a custom text-based protocol.

Core Ideas

- **The Client** (GoGameClient) is the frontend and it provides the visual interface of the game. Players can start a session, play the game, view the board, place stones using mouse clicks, and view a history of moves.
 - **The Server** (GoGameServer) is the backend and it connects players with each other. It waits for 2 players to connect, pairs them into “sessions” and assigns them as Black vs. White , validates moves using the game logic, and shows the same board state across both clients in real-time.
 - **Game Rules:** Players take alternate turns trying to surround each others stones to capture them. The game ends when both players pass , after that captured stones are calculated as points and the winner is declared.
-

2. System Architecture Report

The system is structured as a **Two-Tier Client-Server Application** with an active server and clients.

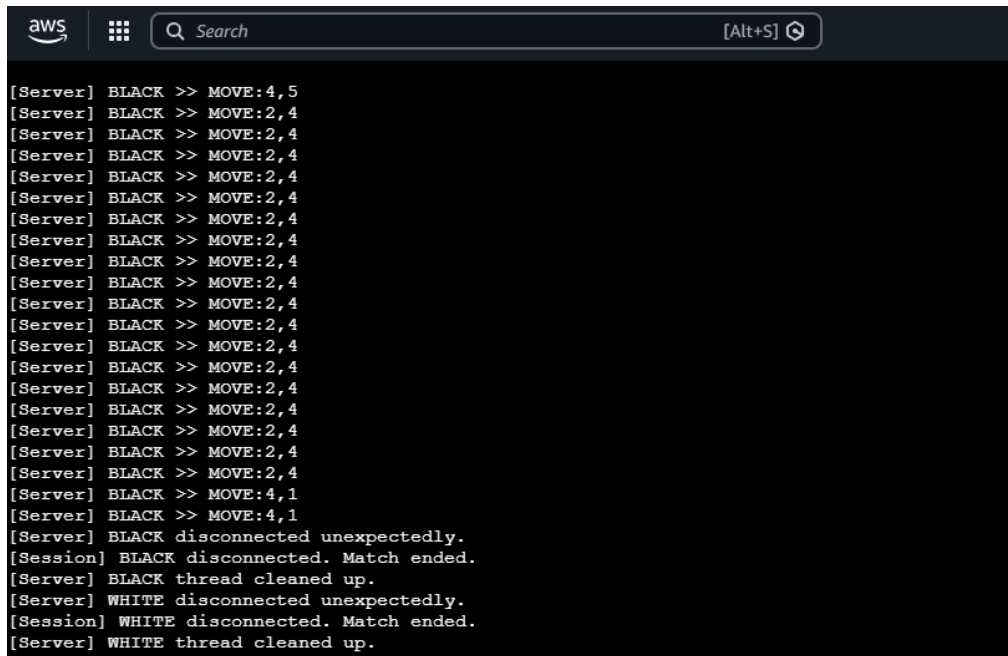
How it works

The system uses a client-server structure where two players connect to a central server hosted in AWS EC2 or locally using TCP communication. Each client has a Java Swing GUI that shows the game board, receives player clicks, and updates the screen during the game. The server connects both players, checks if moves are valid, and keeps track of the game state. The clients and server communicate by sending simple string messages through TCP.

GO GAME PROJECT REPORT

BY ABDEL FATAH M. A. ALHOOT (2321051372)

The server console logs and displays all player moves and connection events in real time, as shown in the figure bellow



```
aws [Search] [Alt+S]
[Server] BLACK >> MOVE:4,5
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:2,4
[Server] BLACK >> MOVE:4,1
[Server] BLACK >> MOVE:4,1
[Server] BLACK disconnected unexpectedly.
[Session] BLACK disconnected. Match ended.
[Server] BLACK thread cleaned up.
[Server] WHITE disconnected unexpectedly.
[Session] WHITE disconnected. Match ended.
[Server] WHITE thread cleaned up.
```

Component Responsibilities

- **Client Responsibilities:** Strictly presentation and user input. The client *does not* calculate captures or determine if a move is legal. It simply forwards user clicks (MOVE:r,c) to the server and redraws the UI when the server sends back a new board state (UPDATE:data).
- **Server Responsibilities:** Session management, connecting players, input validation, and broadcasting state changes.
- **Communication Model:** TCP style commincation. The client has a dedicated background thread (NetworkClient-Listener) to continuously read server broadcasts without blocking the Swing Event Dispatch Thread (EDT).
- **Threading Model (Server):** The GoServer loops to accept incoming sockets. when connection is established, it immediately calls a ClientHandler thread for that socket. When two players connect, their handlers are linked to a shared GameSession.

GO GAME PROJECT REPORT

BY ABDEL FATAH M. A. ALHOOT (2321051372)

3. Code Structure Analysis

Module & Folder Structure

The project is split into two folders, /GoGameClient for Client program and /GoGameServer for server Program

Key Classes

- **GameLogic:** It handles how the game works, and checks liberties (if a stone is surrounded or not) using Depth-First Search algorithm (DFS), and calculates captures for points.
 - **NetworkClient:** A simple connection layer that handles communication between the client and server and allows the UI to react to server messages.
 - **GameSession:** connects two clients into one session to implement the same methods on both screens at the same time.
 - **GameScreen:** The main UI for the game where players can interact with the board and place stones.
-

4. Game Logic Analysis (Go Game)

The GameLogic class provides the whole algorithm for the game to work.

- **Board Representation:** A simple `int[][]` board using constants (EMPTY = 0, BLACK = 1, WHITE = 2).
 - **Liberty Checking:** Uses a recursive DFS (hasLiberty) with a visited array to prevent infinity loops. It accurately checks if a contiguous group of stones has any adjacent empty spaces.
 - **Captures:** when placing a stone, the game checks the 4 squares around stone for enemy stones. If all squares are taken the stone gets captured.
 - **Scoring:** It checks connected empty spaces and gives the points to the player surrounding that area.
-

5. Networking Analysis

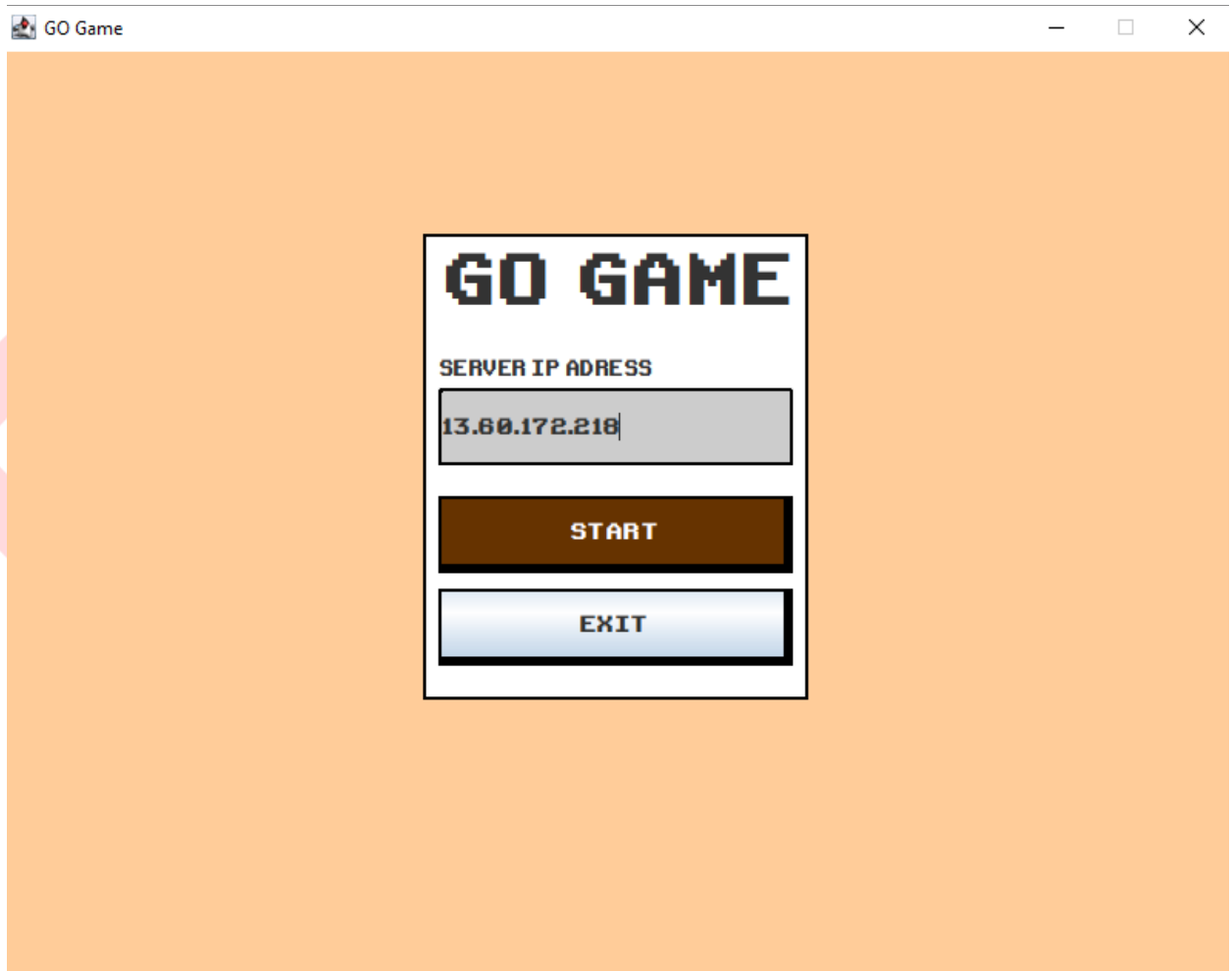
The networking system handles continuous player connections on the server while clients manage proper connect and disconnect actions. Thread safety is ensured by using `SwingUtilities.invokeLater` for UI updates and synchronized methods on the server to protect the game state. The board is sent as a simple 81-character string to keep communication efficient. However, there is an issue where a disconnected waiting player may still appear active because `socket.isClosed()` cannot detect sudden disconnects, which can lead to broken matchmaking.

GO GAME PROJECT REPORT

BY ABDEL FATAH M. A. ALHOOT (2321051372)

6. Screenshots and results

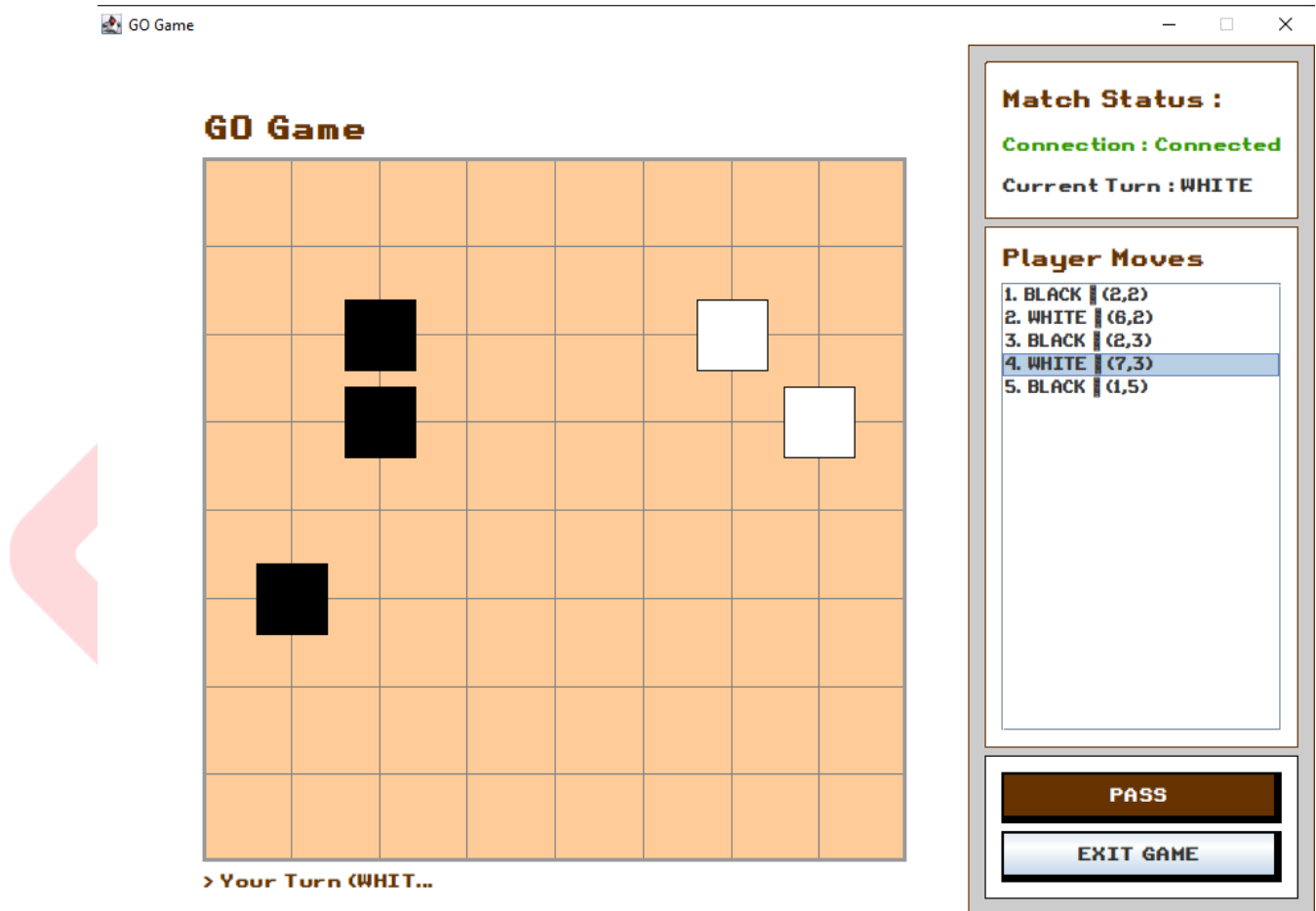
- Start Screen where players can enter the IP address of the server they want and can start a game session. After that it waits for the second player to connect to the server for it to start the game.



GO GAME PROJECT REPORT

BY ABDEL FATAH M. A. ALHOOT (2321051372)

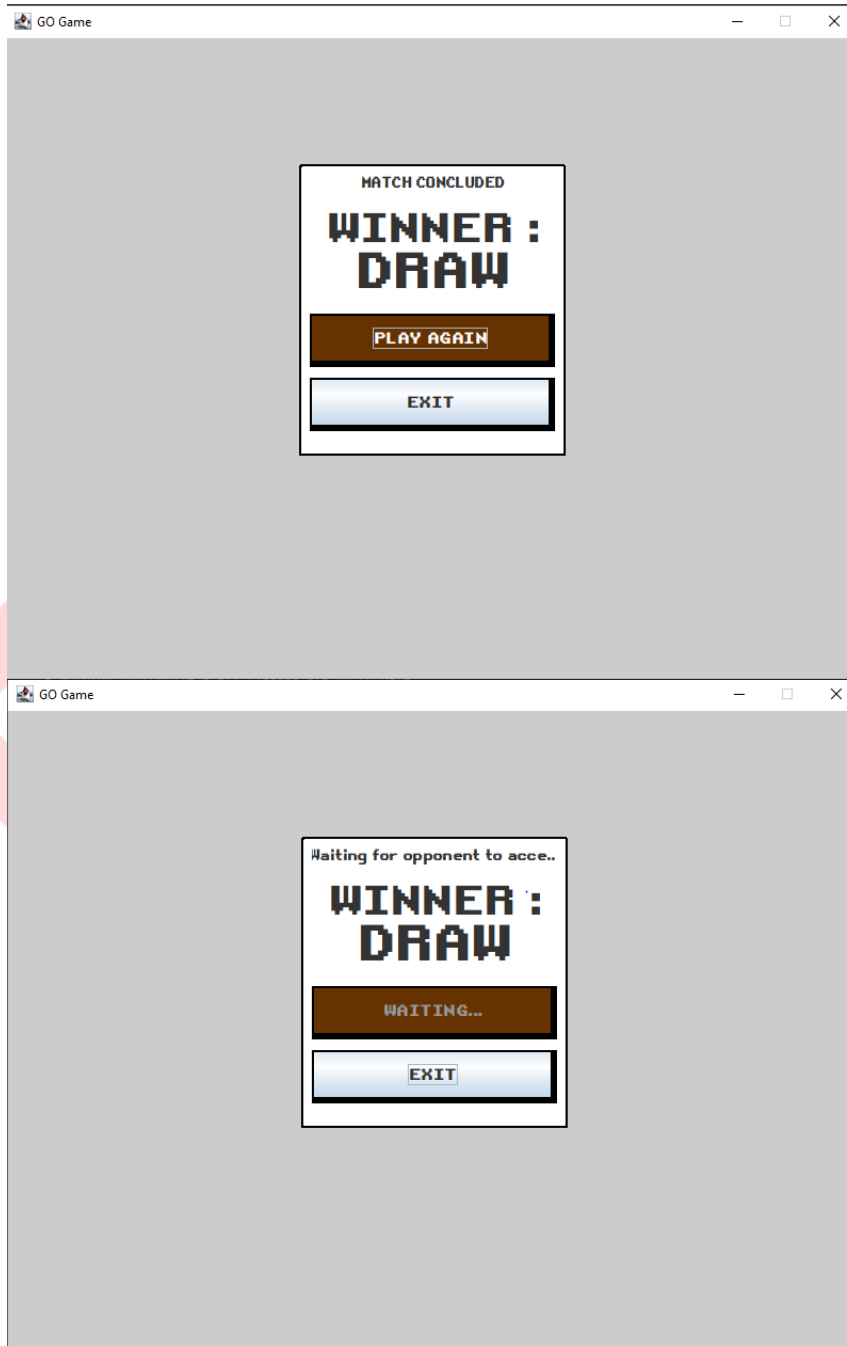
- Game Screen where both players can see the same board, place stones and play with each other in real time and its transmitted to each other through the server



GO GAME PROJECT REPORT

BY ABDELFATAH M. A. ALHOOT (2321051372)

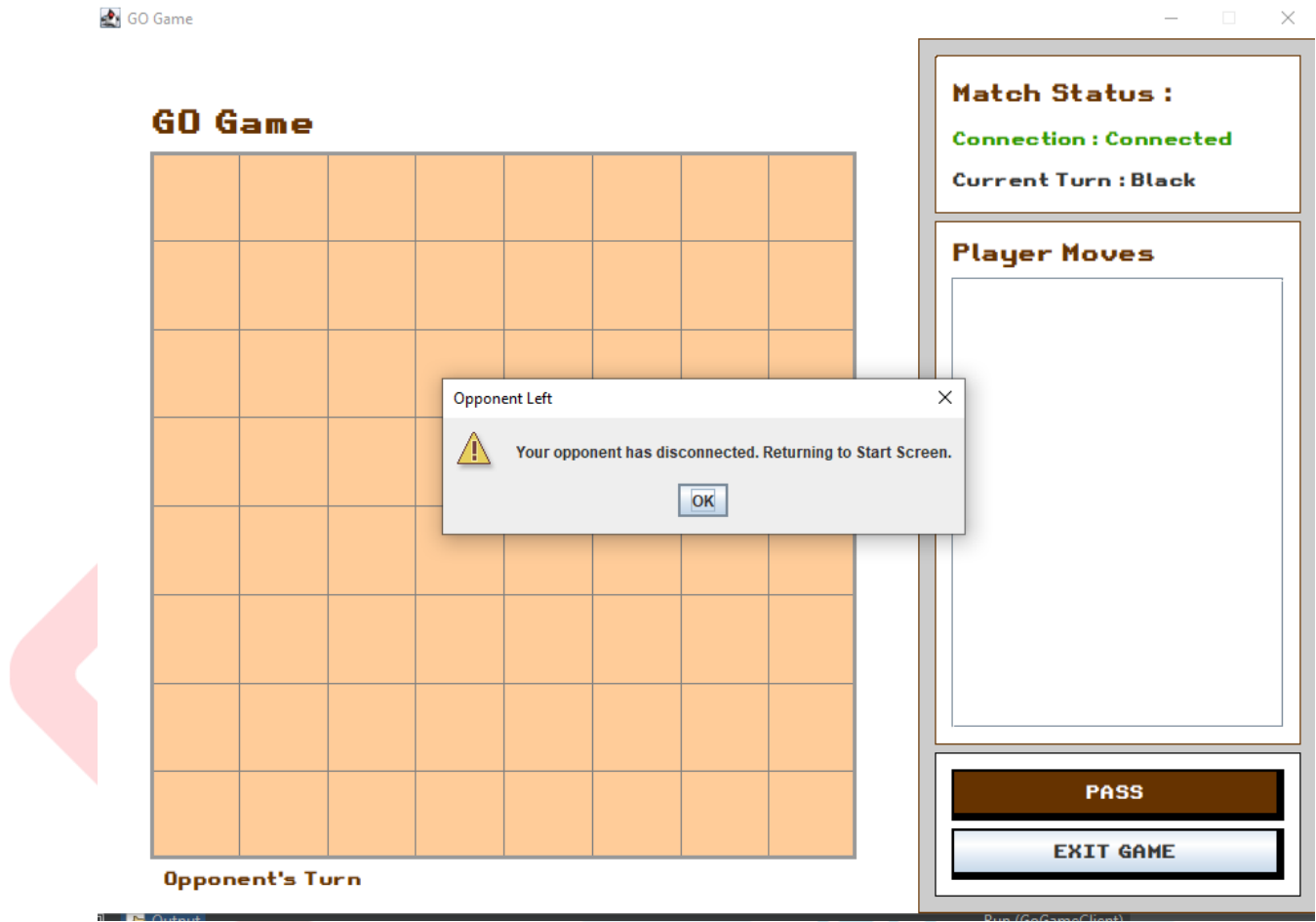
- End Screen where the winner gets announced and players can choose to play again or exit, if play again is chosen the server waits for both players to send requests for it start.



GO GAME PROJECT REPORT

BY ABDEL FATAH M. A. ALHOOT (2321051372)

- When connection is lost or one of the players exits the game, a popup error message will be shown



7. Final Summary

Conclusion: This is a good university-level project that includes GUI, networking, and game logic. The project uses Java Swing for the interface and handles communication between the clients and server correctly using TCP. The Server is hosted on AWS so players are able to connect from anywhere using the IP address provided by AWS.